Workable-Items Model — every WBS task/subtask → the §11.4.93 SQLite single source of truth

Workable-Items Model — every WBS task/subtask → the §11.4.93 SQLite single source of truth

Revision: 3 **Last modified:** 2026-07-04T12:00:00Z **Rev 3:** Independent gap-analysis pass — verified against `06-phase0-spike-wbs.md`, `07/08/09-…-wbs.md`, and the three `subtask-deepening-p*.md` docs; no schema or id-convention contradictions found. §1's R5 reconciliation (Phase-0 fresh-id-per-subtask vs Phase-1..3 dotted `.k` form) confirmed accurate against the now-complete `subtask-deepening-p1/2/3.md` (all three exist, deepen every parent task, and are cited correctly in the `.docs_chain/contexts/wbs.yaml` source list §7).

> Volume 7 (Phase Execution), document 1 of 5. This spec defines the **mechanical contract** by which every Work-Breakdown-Structure leaf in the four phase WBS documents — `06-phase0-spike-wbs.md` (`HVPN-P0-NNN`), `07-phase1-mvp-wbs.md` (`HVPN-P1-NNN`), `08-phase2-parity-wbs.md` (`HVPN-P2-NNN`), `09-phase3-reach-wbs.md` (`HVPN-P3-NNN`) — becomes one row in the git-tracked SQLite single source of truth `docs/workable_items.db` (§11.4.93/.95). It is **spec-only**: it describes the schema, the integrity contract (§11.4.148), the per-item test diary (§11.4.149), the bidirectional docs↔DB sync via Docs Chain (§11.4.106), and the `HVPN-Pn-NNN` id convention (§11.4.54) — it does not build the loader (that is work-item `HVPN-P1-150`). Every claim here is derived from a cited phase WBS doc or constitution anchor; nothing is invented. Effort/

> duration figures inherited from the phase docs are sizing estimates only and are marked `TARGET` where a date could be inferred.

Table of contents

---

0. Why a DB-backed single source of truth

The four WBS documents are human-authored Markdown — readable, reviewable, diffable. But Markdown alone is a fragile tracker: status drifts between the doc and reality, dependency cycles go undetected, and "is this work owed?" has no mechanical answer. §11.4.93 mandates a **SQLite single source of truth** for workable items, and §11.4.95 mandates that DB be **git-tracked** (it is authoritative source data, NOT a build artefact — the §11.4.77 regen-mechanism exemption explicitly does NOT apply).

The model is **bidirectional** (§11.4.93, §11.4.106): the Markdown WBS and the DB are kept byte-identical in a round-trip (`md-to-db` parses the leaves and upserts; `db-to-md` regenerates the leaf blocks). The DB is authoritative; the WBS Markdown, the rendered HTML/PDF, and any external tracker are **derived**. A `verify` pass is the deterministic pre-build gate that fails on any drift (§11.4.50 determinism + §11.4.106 conflict-not-silent-merge).

Each phase WBS already declares this projection in its own `§3 Workable-item schema (§11.4.93 DB-ready)` section: Phase 0 in `06-…` §0.3, Phase 1 in `07-…` §3, Phase 2 in `08-…` §3, Phase 3 in `09-…` §3. This document **reconciles those four into one canonical**

schema and one loader contract so a single DB holds all four phases (the `phase` discriminator column, introduced in `08-…` §3, carries `P0|P1|P2|P3`).

---

1. The id convention (`HVPN-Pn-NNN`, §11.4.54)

Every workable item — task or subtask — carries a stable, unique, auto-incremental ticket id (§11.4.54: append-only, never renumbered, reused, or decremented). HelixVPN uses the `HVPN-` project prefix instead of the generic `ATM-`, but the discipline is identical.

Element	Rule	Source
Prefix	`HVPN-` (project), then phase tag `P0`/`P1`/`P2`/`P3`	all four WBS §0
Numeric block	the epic/milestone encodes the hundreds/tens digit	`07-…` §0, `08-…` §0, `09-…` §0.1
Phase 1+ epics	`Enn` → tasks `nn0..nn9` (E02 store → `020..029`)	`07-…` §0
Phase 0	milestones `S0..S8`; gates use `001..006`-style; cross-cutting `077..080`	`06-…` §0.1
Subtask form	Phase 1+ append `.k` (`HVPN-P1-022.3`); Phase 0 allocates a fresh `HVPN-P0-NNN` per subtask with `parent` expressing hierarchy	`07-…` §0, `06-…` §0.1
Mutability	**monotonic, never renumbered**; gaps allowed; new work appends at next free number	§11.4.54, all four WBS §0
Cross-phase deps	cite the **full** id (`HVPN-P1-090`) — the namespaces are disjoint	`08-…` §0

> **Subtask-id reconciliation (R5, see `REFINEMENT_NOTES.md`).** Phase 0 allocates a *distinct* `HVPN-P0-NNN` for each subtask (e.g. `HVPN-P0-009/010` under task `HVPN-P0-008`). Phases 1–3 use the dotted `.k` form (`HVPN-P1-022.3`). Both are valid §11.4.54 stable ids; the loader (§8) treats the dotted form as a child whose `parent_id` is the bare task id. The three deepening docs in this volume (`subtask-deepening-p1/2/3.md`) author the `.k` subtasks that this model imports.

The `HVPN-Pn-NNN[.k]` string is the **binding key** across all surfaces — the DB `atm_id`/`id` primary key, the WBS heading token, and (when wired) the external tracker custom field (§11.4.148 D1). A heading-hash secondary key (§11.4.54) preserves the binding when wording reflows.

2. The canonical DDL (one table for all four phases)

The four WBS docs declare near-identical `items` + `test_diary` tables; the only divergence is column naming (`id` vs `atm_id`, `est_effort_d` vs `effort_days`) and the Phase-2 `phase` discriminator. The **canonical reconciliation** below is the schema the loader targets; it is a strict superset that round-trips every phase's projection without loss.

```
-- docs/workable_items.db  — git-tracked (§11.4.95), single source
       of truth (§11.4.93).
-- Canonical schema reconciling 06-…§0.3 / 07-…§3 / 08-…§3 / 09-…
       §3.

CREATE TABLE IF NOT EXISTS items (
  atm_id         TEXT PRIMARY KEY,                        -- 'HVPN-
       P1-022' | 'HVPN-P0-009' | 'HVPN-P2-222.1'
  parent_id      TEXT REFERENCES items(atm_id),           -- owning
       epic/milestone/task; NULL at epic root
  phase          TEXT NOT NULL CHECK (phase IN
       ('P0','P1','P2','P3')),
  kind           TEXT NOT NULL CHECK (kind IN
       ('epic','milestone','task','subtask','gate')),
  title          TEXT NOT NULL CHECK (length(title) >=
       6),
       -- DDL char-floor; loader enforces the §11.4.91 ≥6-word/
       ≥40-char rule
  description    TEXT NOT NULL CHECK (length(description) >=
       40),          -- §11.4.91/.148 D2
  status         TEXT NOT NULL DEFAULT 'Queued'
                 CHECK (status IN ('Queued','In progress','Ready
       for testing',
                                  'In
       testing','Reopened','Operator-blocked',
                                  'Obsolete (→
       Fixed.md)','Implemented (→ Fixed.md)',
                                  'Completed (→ Fixed.md)','Fixed
       (→ Fixed.md)')),
  type           TEXT NOT NULL DEFAULT 'Task' CHECK (type IN
       ('Bug','Feature','Task')),
  severity       TEXT NOT NULL DEFAULT 'normal',
  epic           TEXT NOT NULL,
       -- 'E02' | 'S0' | 'E22'
  module         TEXT NOT NULL,                           --
       'store','coordinator','helix-core',…
  gate           TEXT,                                    --
       'G1'..'G6','G20'..'G26' or NULL
  deps           TEXT NOT NULL DEFAULT '[]',              -- JSON
       array of atm_ids (cross-phase allowed)
  deliverable    TEXT NOT NULL,
```

```sql
  acceptance    TEXT NOT NULL,                          --
        falsifiable, captured-evidence (§11.4.5/.69/.107)
  effort_days   REAL NOT NULL DEFAULT 1.0,              -- sizing
        only — never a date (TARGET)
  test_types    TEXT NOT NULL DEFAULT '[]',             -- JSON
        array of §11.4.169 codes
  dod_refs      TEXT NOT NULL DEFAULT '[]',             -- JSON:
        ['AC3','SLO1'] | ['P2-AC3'] | ['G21']
  source_refs   TEXT NOT NULL DEFAULT '[]',             -- JSON:
        ['04_P1 §7','02 §7.2']
  created_at    TEXT NOT NULL DEFAULT (datetime('now')),
  modified_at   TEXT NOT NULL DEFAULT (datetime('now')),

        -- §11.4.148 D1: status + type + id are NOT NULL by
        construction above.
  -- §11.4.104: who created / who owns (canonical participant
        handles; '' = legacy, must still parse).
  created_by    TEXT NOT NULL DEFAULT '',
  assigned_to   TEXT NOT NULL DEFAULT ''
);

-- §11.4.34 / §11.4.93 append-only lifecycle audit (state
        transitions, NOT test runs).
CREATE TABLE IF NOT EXISTS item_history (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  atm_id      TEXT NOT NULL REFERENCES items(atm_id),
  changed_at  TEXT NOT NULL,
  by          TEXT NOT NULL CHECK (by IN
        ('AI','User','Operator')),     -- §11.4.34
  from_status TEXT, to_status TEXT,
  reason      TEXT NOT
        NULL,                                             --
        §11.4.34 closed vocab
  evidence
        TEXT
        -- path/desc (§11.4.7)
);

-- §11.4.21 Operator-blocked detail (WHAT/WHY/UNBLOCK/WHO) — one
        row per blocked item.
CREATE TABLE IF NOT EXISTS operator_block_details (
  atm_id     TEXT PRIMARY KEY REFERENCES items(atm_id),
  what       TEXT NOT NULL, why TEXT NOT NULL,
  unblock    TEXT NOT
        NULL,                                              --
        §11.4.148 D3 enumerated choices
  who        TEXT NOT NULL
);

-- §11.4.90 Obsolete detail (Since/Reason/Superseding/Triple-check
        evidence).
CREATE TABLE IF NOT EXISTS obsolete_details (
  atm_id     TEXT PRIMARY KEY REFERENCES items(atm_id),
  since      TEXT NOT NULL, reason TEXT NOT
        NULL,                       -- §11.4.90 closed vocab
```

```sql
  superseding TEXT, evidence TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS meta (
  schema_version TEXT NOT NULL,
  last_sync_at   TEXT NOT NULL,
  integrity_hash TEXT NOT
        NULL                                    -- §11.4.86
        sha256 of sorted keyset
);

CREATE INDEX IF NOT EXISTS idx_items_parent ON items(parent_id);
CREATE INDEX IF NOT EXISTS idx_items_phase  ON items(phase);
CREATE INDEX IF NOT EXISTS idx_items_gate   ON items(gate);
CREATE INDEX IF NOT EXISTS idx_items_status ON items(status);
```

The `gates` table from `06-…` §0.3 and `09-…` §3 is retained verbatim as a sibling (gate verdicts are not workable items but carry their own evidence path); it is reproduced in §9.

---

3. Field-by-field mapping: WBS leaf → DB row

Each WBS leaf block (a task with no subtasks, or a subtask) carries the field contract declared in every phase doc's `§0 Field dictionary` / `§0 Field contract`. The mapping is 1:1:

WBS field (Markdown)	DB column	Transform / rule
heading token `HVPN-Pn-NNN[.k]`	`atm_id`	verbatim; primary key
owning epic/milestone/task	`parent_id`	the immediate parent's id; epic root → NULL
`Pn` from the id	`phase`	`P0`/`P1`/`P2`/`P3`
`kind` (task/subtask/epic/milestone/gate)	`kind`	from the heading style + §0 hierarchy
`title` (≥6 words, §11.4.91)	`title`	verbatim; loader rejects < 6 words (§11.4.91 anti-pattern)
Desc (≥40 chars, §11.4.148 D2)	`description`	verbatim; loader rejects stub/section-label fragments
(lifecycle)	`status`	default `Queued`; never authored stale in the WBS
`type Task|Feature|Bug`	`type`	

WBS field (Markdown)	DB column	Transform / rule
		from the `· type X` annotation; Phase-2 default `Feature`, else `Task`
(risk)	`severity`	from `Critical`/`normal` annotation where present
epic id (`E02`/`S0`/`E22`)	`epic`	parsed from the id block
`module:` tag	`module`	from the epic header `module:` field
`gate:`/`· DoD:`	`gate`	`G1..G6`/`G20..G26` or NULL
Deps	`deps`	JSON array; – → `[]`; cross-phase ids allowed
Deliverable	`deliverable`	verbatim file paths / artefacts
Acceptance	`acceptance`	verbatim; MUST be a falsifiable captured-evidence assertion
Effort (`XS(1)…XL(15)` / `est_effort` days)	`effort_days`	T-shirt midpoint → REAL; **sizing only, never a date** (`TARGET`)
Tests	`test_types`	JSON array of §11.4.169 codes (`UNIT`,`INT`,…)
`· DoD: AC3` / `· SLO1`	`dod_refs`	JSON array
`[04_P1 §7]` inline cites	`source_refs`	JSON array
(attribution)	`created_by`/ `assigned_to`	§11.4.104 handles; legacy `''` still parses

The §11.4.169 test-type code map (one canonical set + documented aliases). `test_types` stores canonical §11.4.169 codes. The four WBS docs accreted two shorthand dialects — Phase 0's terse abbreviations (`UT/IT/CONC/CH/HQA/SC`, e.g. the `HVPN-P0-008` sample row and the §10 worked example) and the long forms used by Phases 1–3 (`UNIT/INT/CHAOS/CHAL/SCALE/STRESS/…`). The loader normalizes the Phase-0 shorthands to the canonical code on `md-to-db`; both render identically. Canonical codes and their §11.4.169 type (Phase-0 alias in parentheses where it differs):

Canonical code	§11.4.169 test type	P0 alias
`UNIT`	unit	`UT`
`INT`	integration	`IT`
`E2E`	e2e	`E2E`

Canonical code	§11.4.169 test type	P0 alias
`FA`	full-automation	`FA`
`CHAL`	Challenges (`challenges` submodule)	`CH`
`HQA`	HelixQA (test banks + autonomous sessions)	`HQA`
`DDOS`	DDoS / load-flood	`DDOS`
`SEC`	security	`SEC`
`CHAOS`	stress + chaos (chaos half)	—
`STRESS`	stress (load/contention/boundary)	—
`CONC`	concurrency / atomicity	`CONC`
`RACE`	race-condition / deadlock	—
`MEM`	memory (leak / soak / ceiling)	—
`BENCH`	benchmarking / performance	`BENCH`
`SCALE`	scaling	`SC`
`UX`	ux	`UX`
`REC`	recorded media evidence (§11.4.158/.159)	—
`REPRO`	reproducibility (bit-identical builds)	—

The only genuine alias rewrites are `UT→UNIT`, `IT→INT`, `CH→CHAL`, `SC→SCALE`; `CONC`, `HQA`, `BENCH`, `FA`, `E2E`, `UX` are already identical across phases. `validate` rejects any code outside this canonical set (or its documented alias).

Effort honesty (§11.4.6). Every phase WBS states explicitly that effort roll-ups are "indicative person-day totals … **not** a commitment" (`07-…` §22, `08-…` §19, `09-…` §14). The loader stores `effort_days` as a sizing scalar; any report deriving a *date* from it MUST label the date `TARGET` and cite the no-commitment caveat. Phase-3 device-gated items carry the widest error bars (`09-…` §14) and are additionally `UNCONFIRMED:/PENDING_DEVICE:` (§11.4.6).

---

4. Sample rows — one per phase, drawn from the WBS docs

The `INSERT` form proves the projection is unambiguous. These rows are transcribed from the exact leaves in the WBS docs (not invented).

Phase 0 leaf — `HVPN-P0-008` (plain-UDP transport baseline, `06-…` §3):

```sql
INSERT INTO items
        (atm_id,parent_id,phase,kind,title,description,type,severity,

        epic,module,gate,deps,deliverable,acceptance,effort_days,test_types,
                   dod_refs,source_refs,created_at,modified_at)
VALUES (
 'HVPN-P0-008','HVPN-P0-S0','P0','task',
 'Implement plain-UDP Transport and prove a loopback echo round-
        trip',
 'Trivial PlainUdp over tokio UdpSocket giving the throughput
        baseline every other transport''s ≥50% bar (G2) is
        computed against; loopback echo validates the trait round-
        trips.',
 'Feature','normal','S0','helix-transport','G1',
 '["HVPN-P0-006"]',
 'helix-transport/src/plain_udp.rs + tests/loopback_echo.rs',
 'echo test sends 10k 1280-byte datagrams loopback, 0 lost, order-
        independent; captured CSV of round-trip count',
 0.5,'["UT","IT","CONC","BENCH"]','["G1"]','["04_P0 §4.3"]',
 '2026-06-26T12:00:00Z','2026-06-26T12:00:00Z');
```

Phase 1 leaf — `HVPN-P1-022` (FORCE RLS, `07-…` §7 / §3 example):

```sql
INSERT INTO items
        (atm_id,parent_id,phase,kind,title,description,type,severity,

        epic,module,gate,deps,deliverable,acceptance,effort_days,test_types,
                   dod_refs,source_refs,created_at,modified_at)
VALUES (
 'HVPN-P1-022','HVPN-P1-E02','P1','task',
 'FORCE RLS + tenant_isolation policies on all 10 tenant tables',
 'Enable + FORCE row-level security with a tenant_isolation USING/
        WITH CHECK policy on every tenant-scoped table and abort
        startup if the app role can bypass RLS.',
 'Task','high','E02','store',NULL,
 '["HVPN-P1-021"]',
 'migrations/0002_rls.sql; store/rls_guard.go startup check',
 'a crafted cross-tenant read returns ZERO rows (captured);
        startup aborts when role has rolsuper/rolbypassrls; CHAOS
        drop mid-tx rolls back, no leak.',
 4,'["UNIT","INT","SEC","CHAOS"]','["AC8","AC2","AC3"]','["02
        §2.3","04_P1 §2.2"]',
 '2026-06-26T12:00:00Z','2026-06-26T12:00:00Z');
```

Phase 2 leaf — `HVPN-P2-222` (UDP hole punching, `08-…` §8 / §3 example):

```sql
INSERT INTO items
        (atm_id,parent_id,phase,kind,title,description,type,severity,

        epic,module,gate,deps,deliverable,acceptance,effort_days,test_types,
                   dod_refs,source_refs,created_at,modified_at)
VALUES (
 'HVPN-P2-222','HVPN-P2-E22','P2','task',
 'UDP hole punching across the NAT-type matrix',
```

```
 'Drive simultaneous WG handshake probes to every peer candidate
        so full-/restricted-/port-restricted-cone NATs open a
        direct path, latching WireGuard roaming onto the working
        endpoint; symmetric pairs fall through to relay.',
 'Feature','high','E22','helix-core',NULL,
 '["HVPN-P2-220","HVPN-P2-221"]',
 'helix-core/src/nat/punch.rs; netns NAT-matrix harness',
 'direct datagrams captured bypassing the gateway for full/
        restricted/port-restricted pairs; symmetric pairs cleanly
        fall through to relay; no false direct claim
        (§11.4.107).',
 8,'["UNIT","E2E","STRESS","SEC","CHAL"]','["P2-AC3","P2-
        SLO1"]','["04_P2 §3.2","SYNTHESIS §3 D6"]',
 '2026-06-26T12:00:00Z','2026-06-26T12:00:00Z');
```

Phase 3 leaf — `HVPN-P3-211` (HarmonyOS VPN ability, `09-…` §6 / §3 seed):

```
INSERT INTO items
        (atm_id,parent_id,phase,kind,title,description,type,severity,

        epic,module,gate,deps,deliverable,acceptance,effort_days,test_types,
                   dod_refs,source_refs,created_at,modified_at)
VALUES (
 'HVPN-P3-211','HVPN-P3-E21','P3','task',
 'HarmonyOS Network Kit VpnExtensionAbility + lifecycle + kill-
        switch wiring',

        'ArkTS VpnExtensionAbility opens the Network Kit tunnel
        fd, hands it to the helix-core .so over NAPI, and drives
        connect/disconnect/kill-switch from the core status
        stream; proven by tunnel UP carrying traffic on a real
        device.',
 'Feature','Critical','E21','shims/harmonyos','G21',
 '["HVPN-P3-210","HVPN-P3-201"]',
 'shims/harmonyos/entry/src/main/ets/HelixVpnAbility.ets + napi
        bridge',
 'PENDING_DEVICE: on a HarmonyOS NEXT device, enroll->UP->curl
        reaches authorized LAN host; window-scoped MP4 + DevEco
        heap capture; §11.4.3 SKIP hardware_not_present until G20
        provisions a device.',
 8.0,'["UNIT","INT","E2E","MEM","UX","REC","CHAL"]','["G21"]','["04_UI
        §6.1","04_ARCH §5.7"]',
 '2026-06-26T12:00:00Z','2026-06-26T12:00:00Z');
```

---

5. The §11.4.148 integrity contract (status + type + id, everywhere)

§11.4.148 binds the workable-item discipline into one DB↔docs↔tracker integrity contract. The five clauses, instantiated for HelixVPN:

- **D1 — no item without a valid status + type + id, on ALL surfaces.** The DDL enforces `atm_id` (PK, NOT NULL), `status` (NOT NULL, closed CHECK set), and `type` (NOT NULL, closed CHECK set). The loader rejects a WBS leaf missing any of the three; the `db-to-md` regen always emits all three; the external tracker push (D5) carries all three. The id is the cross-surface binding key.
- **D2 — comprehensive structured description.** `description` ≥ 40 chars (§11.4.91) carrying WHAT + HOW-it-manifests + acceptance intent. The *reproduce* and *acceptance-criteria* facets live in `acceptance` (a falsifiable captured-evidence assertion per §11.4.5/.69/.107). The loader rejects §11.4.91 anti-pattern fragments (`Composes with`, bare `Critical`, a §-letter alone).
- **D3 — BLOCKED items carry WHY + enumerated unblock CHOICES.** Any `Operator-blocked` item (§11.4.21) MUST have an `operator_block_details` row enumerating the closed list of unblock choices (`[A]…·[B]…·[C]…`, §11.4.66 shape). Phase 3 is the heaviest user: `HVPN-P3-211/221` carry `unblock = "provision a HarmonyOS NEXT / Aurora device + add to the §11.4.128 tracked-device set"`; `HVPN-P3-252` (audit engagement) carries `unblock = "fund + contract an independent audit firm (§11.4.101 spend block)"`.
- **D4 — never-missed bidirectional DB↔docs↔tracker sync** — §7 below.
- **D5 — generic idempotent external-tracker push** — statuses (collapsed onto the tracker's native set per §11.4.33/.112, precise value preserved in a header), types, assignee (§11.4.104 handle from a project env var, never hardcoded/logged §11.4.10), and per-item sub-tasks; match-by-stable-key `[HVPN-Pn-NNN]`, present⇒UPDATE/absent⇒CREATE, sink-side `created=N updated=M failed=0` proof (§11.4.69). The tracker, list/board id, and field map are consumer-registered at runtime (§11.4.28 decoupling), never hardcoded.

A release tag is blocked while any item violates D1–D3 (`07-…` §3 anti-bluff note; §11.4.148 release-blocker clause).

---

6. The §11.4.149 per-item test diary

§11.4.149 mandates an append-only **testing diary** — one row per test *run* against an item — distinct from `item_history` (lifecycle state transitions). The schema (already declared in `07-…` §3 and `08-…` §3, restated canonically):

```
CREATE TABLE IF NOT EXISTS test_diary (
  id             INTEGER PRIMARY KEY AUTOINCREMENT,
  atm_id         TEXT NOT NULL REFERENCES items(atm_id),
  date_time      TEXT NOT NULL,
        -- ISO-8601 UTC
  tested_by      TEXT NOT NULL CHECK (tested_by IN
        ('User','Operator','AI-agent','HelixQA')),
  result         TEXT NOT NULL CHECK (result IN
        ('PASS','FAIL','SKIP')),  -- §11.4.45 vocab
  test_type      TEXT NOT NULL,
        -- one §11.4.169 code
  evidence_path TEXT,
        -- qa-results/<run-id>/…
  feature_class TEXT,
        -- §11.4.69 sink-side taxonomy class
  action_taken  TEXT NOT NULL DEFAULT '',
  status_changed TEXT NOT NULL DEFAULT '',
        -- from→to if this run flipped status
  observations  TEXT NOT NULL,
        -- in-depth, facts or UNCONFIRMED: (§11.4.6)
  -- THE anti-bluff constraint: a PASS row is impossible without
        captured evidence.
  CHECK (result <> 'PASS' OR (evidence_path IS NOT NULL AND
        length(evidence_path) > 0))
);
```

The `CHECK (result <> 'PASS' OR evidence_path …)` clause makes a PASS-bluff *mechanically impossible at the schema layer*: SQLite itself rejects a PASS row with no evidence path (`07-…` §3, §11.4.149(a)). The diary feeds §11.4.132 risk-ordered validation (most-recently-tested / most-reopened first) and the per-item `Diary/Diary_Summary` four-format exports (§11.4.65, §11.4.149(c)).

Tooling is minimal-LLM and deterministic (§11.4.149(e)): the `observations` prose is authored by whoever ran the test; the tooling only stores / renders / pushes / validates. The external-tracker SUB-TASK lifecycle (`TODO|In-progress|Completed`, §11.4.149(d)) reuses the D5 idempotent push.

---

7. Docs↔DB sync via Docs Chain (§11.4.106)

The WBS Markdown ⇄ DB ⇄ exports round-trip is mechanized by **Docs Chain** (`vasic-digital/docs_chain`, §11.4.106) — the canonical bidirectional document-and-database propagation engine, consumed **by reference** (never copied) and configured as data via `.docs_chain/contexts/wbs.yaml` (`07-…` §20 HVPN-P1-150, `08-…` §16 HVPN-P2-300, `09-…` §12 HVPN-P3-272).

```yaml
# .docs_chain/contexts/wbs.yaml  (consumer-owned, §11.4.28 decoupling)
context: helixvpn-wbs
sources:
  - path: docs/research/mvp/final/06-phase0-spike-wbs.md     # leaf blocks → items(phase=P0)
  - path: docs/research/mvp/final/07-phase1-mvp-wbs.md       #             → items(phase=P1)
  - path: docs/research/mvp/final/08-phase2-parity-wbs.md    #             → items(phase=P2)
  - path: docs/research/mvp/final/09-phase3-reach-wbs.md     #             → items(phase=P3)
  - path: docs/research/mvp/final/v07-execution/subtask-deepening-p1.md  # .k subtasks
  - path: docs/research/mvp/final/v07-execution/subtask-deepening-p2.md
  - path: docs/research/mvp/final/v07-execution/subtask-deepening-p3.md
db:    docs/workable_items.db                                  # git-tracked (§11.4.95)
exports: [html, pdf]                                           # §11.4.65 siblings; DOCX added for Status set (§11.4.153)
change_detection: content-hash                                 # §11.4.86 (NOT mtime)
on_conflict: surface                                           # §11.4.6 — never silent-merge
evidence_dir: qa-results/docs_chain/<run-id>/                  # §11.4.69 captured per run
```

Docs Chain guarantees (§11.4.106): content-hash change detection (§11.4.86, not mtime), atomic-rename + SQLite-txn commit + rollback (§9.2), both-dirty `sync` → conflict-surfaced-not-merged (§11.4.6), `verify` as the deterministic pre-build gate (§11.4.50), per-run captured evidence (§11.4.69), and an honest typed `ToolAbsentError` + §11.4.3 SKIP when pandoc/weasyprint is missing (never a fake transform). The `meta.integrity_hash` (sha256 of the sorted item keyset, §11.4.86) is the drift-proof fingerprint the freshness gate compares.

---

8. The loader contract (`cmd/workable-items/`, HVPN-P1-150)

A deterministic Go binary (NOT LLM-driven in the data path, §11.4.149(e)) implements the projection. It lives in the constitution submodule for cross-project reuse (§11.4.74, §11.4.93). Subcommands:

Subcommand	Behaviour
`md-to-db`	parse every WBS leaf block → upsert `items`/`item_history` by `atm_id`; reject §11.4.91/.148-D1/D2 violations
`db-to-md`	regenerate the leaf blocks from the DB — **byte-identical round-trip** (§11.4.93, closed-set whitespace/section-order tolerance)
`diff`	show md↔db divergence; non-empty diff blocks commit (`commit_all.sh` refuses, §11.4.93)
`validate`	run in the pre-build sweep — enforce id uniqueness/monotonicity (§11.4.54), dependency-DAG acyclicity, status/type closed sets, the §11.4.149 PASS-needs-evidence constraint
`add` / `close`	structured mutation that stages+commits+pushes the DB alongside the MD regen (§11.4.95, §2.1 multi-upstream) with a WAL checkpoint before stage

The loader is itself 100% test-covered (§11.4.27): unit (parse/regen), integration (against a real SQLite + a real external tracker with an honest §11.4.3 SKIP when the token is absent), the byte-identical round-trip property test (§11.4.93), export, a HelixQA Challenge (`CME-WORKABLE-ITEMS-001`-class), and a paired §1.1 mutation (corrupt a row → `validate` FAILs).

---

9. Gates, anti-bluff, and what "complete" means

The `gates` sibling table (verbatim from `06-…` §0.3 + `09-…` §3) tracks the phase exit gates that unblock/close work, with their own evidence path:

```
CREATE TABLE IF NOT EXISTS gates (
  id            TEXT PRIMARY KEY,                        --
       'G1'..'G6','G20'..'G26'
  question      TEXT NOT NULL,
  go_no_go_bar  TEXT NOT NULL,
```

```
  section        TEXT NOT NULL,
  owning_epic    TEXT,
  outcome        TEXT NOT NULL DEFAULT 'pending'
                 CHECK (outcome IN
        ('pending','pass','fail','rust','go',

        'pending_device','pending_toolchain','operator_blocked')),
  evidence_path TEXT
);
```

A leaf is `complete` only when (`07-…` §0 anti-bluff note, §11.4.93/.149):

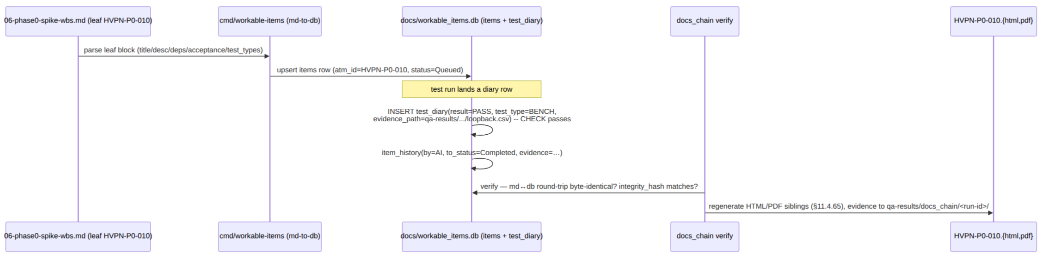
1. its required §11.4.169 `test_types` are all green **with** a `test_diary` row per type carrying a non-empty `evidence_path` (the schema CHECK enforces this for PASS rows);
2. where it has a user-visible surface, a window-scoped MP4 (§11.4.154/.155) vision-verified through the §11.4.163 media-validation pipeline exists;
3. its lifecycle reached a terminal `status` (`Fixed/Implemented/Completed (→ Fixed.md)`) with an `item_history` closure row (§11.4.34 By/Reason/Evidence);
4. its gate (if any) has `outcome IN ('pass','rust','go')` — or, for Phase-3 device-/engagement-/toolchain-gated items, an honest `pending_device/pending_toolchain/operator_blocked` (e.g. G26 reproducibility held `PENDING_TOOLCHAIN:` per `09-…` R-P3-7) with the §11.4.148 D3 unblock condition, **never** a faked pass (§11.4.6, `09-…` §13 honest-gap rule).

Metadata-only / config-only / grep-without-runtime PASS is forbidden (§11.4 / §11.4.1). The §11.4.147 agent registry treats any non-`complete` item as work still owed; the §11.4.126 endless-loop done-condition cannot read satisfied while any item is non-terminal.

---

10. Worked end-to-end example (P0 leaf → row → diary → export)

Tracing `HVPN-P0-010` (loopback echo soak, `06-…` §3) through the whole model:



1. The leaf in `06-…` §3 declares `acceptance: 0 lost @ loopback; deterministic over N=3 (§11.4.50)` and `test_types: [IT, CONC, BENCH, FA]`.
2. `md-to-db` upserts the `items` row with `status='Queued'`, `phase='P0'`, `parent_id='HVPN-P0-008'`.
3. A real run captures `qa-results/<run-id>/loopback_echo.csv`; a `test_diary` row records `result='PASS', test_type='BENCH', evidence_path=…` — the schema CHECK admits it *only because* the evidence path is non-empty.
4. The N=3 determinism (§11.4.50) lands three identical-hash diary rows; an `item_history` row flips `status → 'Completed (→ Fixed.md)'`.
5. `docs_chain verify` confirms the md↔db round-trip is byte-identical and the `meta.integrity_hash` matches, then regenerates the HTML/PDF siblings.

The same flow applies to every leaf across all four phases; the three deepening docs in this volume author the `.k` subtask leaves that feed it.

Sources verified

- `06-phase0-spike-wbs.md` (Phase 0 WBS, §0.3 DDL, §0.4 test-type map, gates table) — read 2026-06-26.
- `07-phase1-mvp-wbs.md` (§3 items/test_diary DDL + sample row, §0 field contract, §20 HVPN-P1-150 loader, §22 effort caveat) — read 2026-06-26.
- `08-phase2-parity-wbs.md` (§3 DDL with `phase` discriminator + sample row, §16 HVPN-P2-300) — read 2026-06-26.
- `09-phase3-reach-wbs.md` (§3 DDL + seed row, §12 HVPN-P3-272, §13 honest-gap rule, §14 risk register) — read 2026-06-26.
- `REFINEMENT_NOTES.md` R5 (subtask-tier asymmetry / DB import driver) — read 2026-06-26.
- Constitution `CLAUDE.md` anchors §11.4.34/.54/.65/.74/.86/.90/.91/.93/.95/.104/.106/.132/.147/.148/.149/.169 — project context, read 2026-06-26.

> Honest boundary (§11.4.6): this document specifies the model and the loader contract; it does **not** assert the loader is implemented (that is the open work-item `HVPN-P1-150`). All effort/duration figures are sizing-only `TARGET`s per the phase docs' no-commitment caveat.